

우당탕 방범대

물리 엔진과 NavMesh 없이 대량 몬스터를 직접 관리한 3D 뱀서라이크

귀여운 캐릭터로 몬스터 무리를 쓸어내며 성장하고, 장애물을 돌파해 보스를 공략하는 캐주얼 서바이벌 액션

플레이영상 GitHub Portfolio

개발 기간
26.05 ~ 26.06 (1개월)

팀 구성
4인 (개발 4)

담당 역할
팀장 · 코어 시스템 · 전투 판정

개발 환경
Unity 6 · URP · C# / PC

프로젝트 하이라이트



플레이어를 추적하는 몬스터 무리



보스 공격 패턴 회피



레벨업과 캐릭터 성장



장애물 돌파

핵심 구현

① 풀링과 ManagedUpdate

생성·파괴와 개별 Update를 없애고 매니저가 일괄 갱신

② 그리드 공간 분할

물리 충돌 없이 인접 셀만 검사해 겹침 방지와 통과 방지 처리

③ NavMesh 없는 3D 추적

단차별로 Ray를 골라 쏘고, 플레이어와 가까운 층을 선택

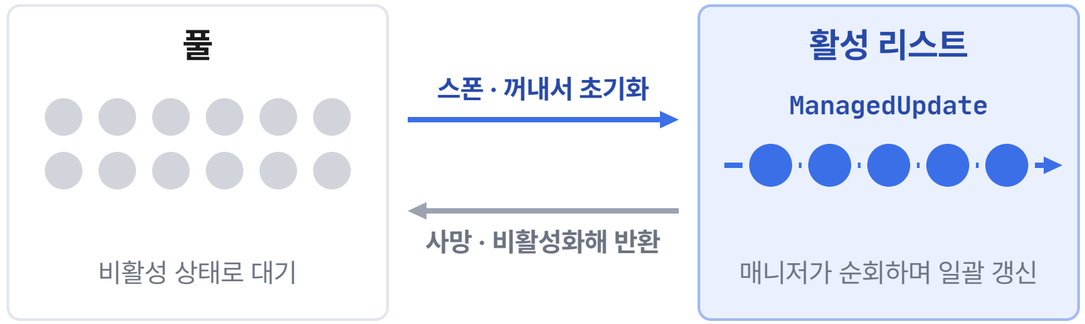
④ VAT + GPU Instancing

애니메이션을 텍스처로 구워 Animator 없이 렌더링

우당탕 방범대 | 핵심 구현 ① 풀링과 ManagedUpdate

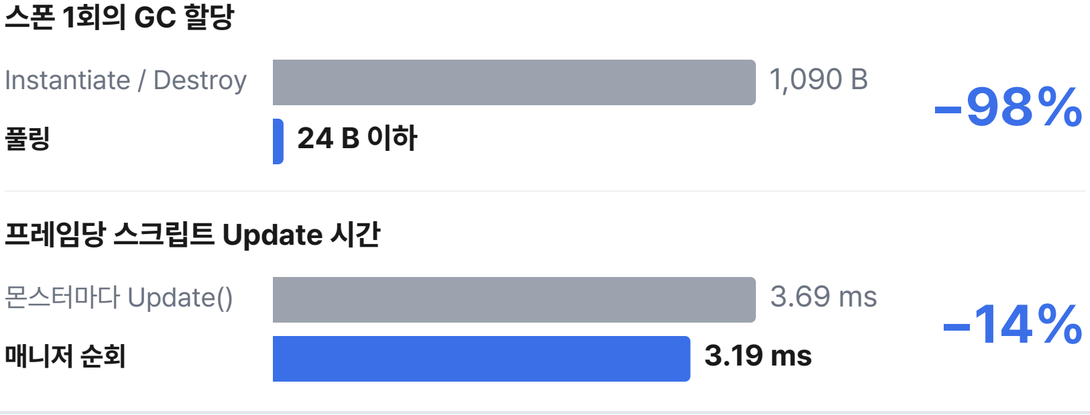
생성·파괴와 개별 Update를 없앤 매니저 일괄 갱신

구조 **풀 재사용과 매니저 일괄 갱신**



생성·파괴 없이 풀에서 재사용하고, 매니저 루프 하나가 전체 몬스터를 갱신

측정 **기본 방식과 같은 조건에서 비교 측정**



풀링과 매니저 일괄 갱신 적용 전후의 GC 할당·Update 시간 비교

설계 목표 - 생성·파괴와 개별 Update 비용 제거

- 스폰과 사망이 반복돼도 Instantiate/Destroy와 GC 할당이 없는 구조
- 몬스터 수만큼 일어나는 Unity 메시지 호출을 루프 하나로 통합

구현 내용 - 풀 재사용 + 매니저의 ManagedUpdate 호출

- 스폰은 풀에서 꺼내 상태만 초기화, 사망은 비활성화해 풀로 반환
- 풀은 지정한 최대 한도까지만 생성하고, 넘으면 스폰 생략
- 몬스터 수만큼 일어나는 Update() 호출을 매니저 루프 하나로 통합

검증 및 효과

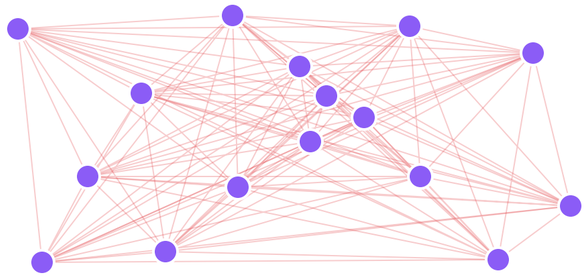
- **풀 재사용으로 스폰 1회의 GC 할당 98% 감소**
- **매니저 순회로 프레임당 스크립트 Update 시간 14% 감소**

우당탕 방범대 | 핵심 구현 ② 그리드 공간 분할

물리 충돌 없이 인접 셀만 검사하는 몬스터 간 겹침 방지

기본 방식 **Rigidbody 충돌 — 밀집할수록 접촉 쌍과 충돌 계산이 급증**

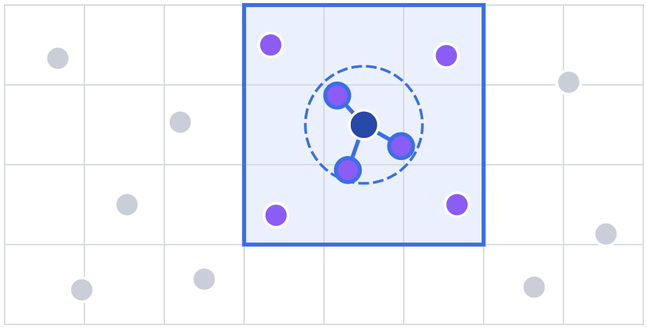
n마리 → 최대 n(n−1)/2 쌍
밀집할수록 급증



물리 엔진에 맡기면 접촉한 쌍마다 충돌을 계산

현재 구조 **그리드 분리 — 자기 셀과 인접 8셀(3×3)만 검사**

검사 대상은 3×3 = 9셀
힘 계산은 최대 5마리까지



같은 셀과 인접 셀의 몬스터만 검사하고, 그리드는 0.15초마다 갱신

설계 목표 - 물리 충돌 대신 겹침만 막는 가벼운 분리

- 몬스터끼리 정확한 충돌은 필요 없고, 과하게 겹치는 것만 막으면 충분
- 물리 엔진에 맡기지 않고 가까운 몬스터만 검사하는 근사 분리로 처리

구현 내용 - 주기적 셀 등록 + 매 프레임 분리 힘 계산

- 일정 시간마다 모든 몬스터를 순회해 같은 셀끼리 셀 좌표 Key로 관리
- 매 프레임 몬스터마다 자기 셀과 인접 8셀에 등록된 이웃을 순회
- 반경 안 이웃 최대 5마리까지, 멀어지는 방향을 거리에 반비례해 합산
- 합산한 힘을 추격 속도에 더해 위치를 직접 밀지 않고 서서히 벌림

검증 및 효과

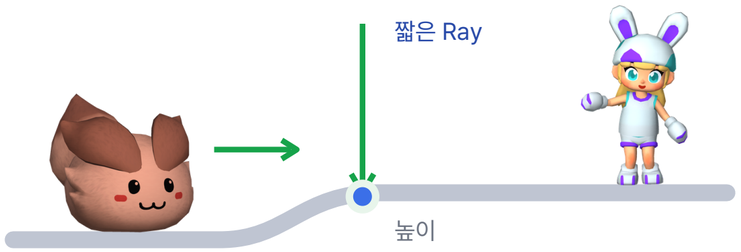
- **물리 엔진에 의존하지 않고 몬스터끼리 겹침 방지 구현**
- **300마리의 겹침 방지·추격·지면 검사를 프레임당 3.0 ms에 처리**

우당탕 방범대 | 핵심 구현 ③ NavMesh 없는 3D 추적

수평은 직선 추적, 수직은 Ray로 푸는 다층 지형 이동

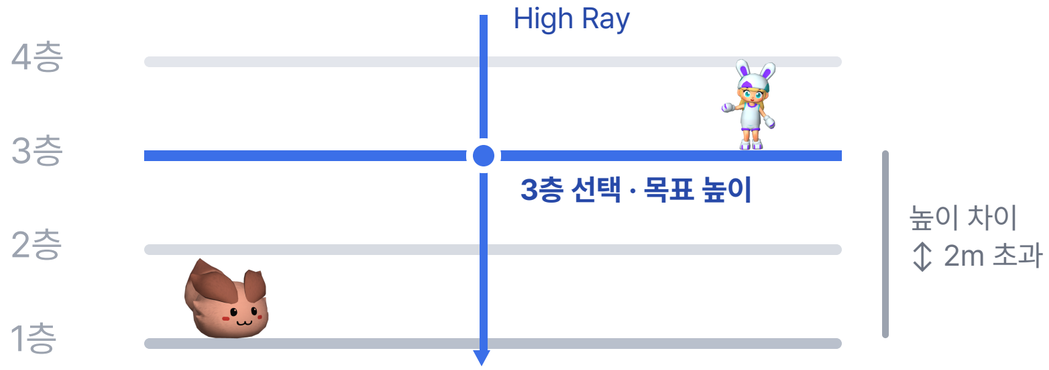
Low **일반 지면 검사 · 단일 결과 Raycast**

평지·작은 언덕에서 짧은 Ray로 지면 높이 확인



Low Ray로 확인한 지면 높이에 맞춰 이동

High **다층 지면 검사 · 다중 결과 Raycast**



1층 몬스터 → 플레이어가 있는 3층을 목표 높이로 선택

설계 목표 - 경로 탐색 없이 다층 지형을 따라가는 추적

- 몬스터마다 경로를 계산하지 않고 플레이어 방향으로 곧장 이동
- 층이 겹친 지형에서도 플레이어 높이에 가까운 지면을 선택해 추적

구현 내용 - 수평 직선 이동 + 단일·다중 결과 Raycast로 지면 추적

- 수평은 플레이어 방향으로 XZ 직선 이동, 경로 탐색 없음
- 평지와 작은 언덕에서는 짧은 단일 결과 Raycast로 높이를 확인해 이동
- 플레이어가 몬스터보다 2m 넘게 높거나 단일 검사 실패 시 다중 검사 실행
- 검출한 지면 중 플레이어 높이에 가장 가까운 층을 선택해 이동

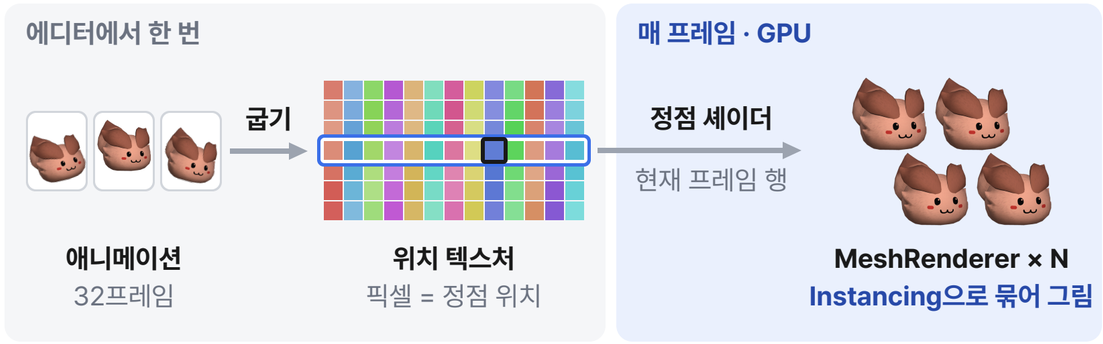
검증 및 효과

- **NavMesh 베이크와 에이전트 없이 다층 지형 추적**

우당탕 방범대 | 핵심 구현 ④ VAT + GPU Instancing

애니메이션을 텍스처로 구워 Animator 없이 렌더링

VAT 애니메이션을 위치 텍스처로 굽고, 셰이더가 읽어 재생



본 계산과 스키닝은 굽는 단계로 옮기고, 매 프레임에는 텍스처만 읽음

측정 같은 모델 1,000마리 · 렌더 스케일 0.3에서 비교 측정

프레임당 메인 스레드 시간

Animator	26.8 ms	−76%
VAT	**6.4 ms**	

몬스터 드로우콜 빈 씬 대비 증가분

Animator	5,121개	−99.6%
VAT	**21개**	

VAT 적용 전후의 메인 스레드 시간·드로우콜 비교

설계 목표 - 몬스터 수만큼 늘어나는 애니메이션 비용 제거

- 몬스터마다 도는 Animator 없이 애니메이션을 재생하는 구조
- 몬스터 수가 늘어도 CPU 시간과 드로우콜이 함께 늘지 않는 구조

구현 내용 - Vertex Animation Texture(VAT) 도입

- SmartVAT로 일반 몬스터의 프레임별 정점 위치를 텍스처에 굽기
- 정점 셰이더(ShaderGraph)가 프레임 행을 차례로 읽어 애니메이션 재생
- 같은 종류끼리 GPU Instancing으로 묶어 드로우콜 감소

검증 및 효과

- **메인 스레드 시간 76% 감소**
- **몬스터 드로우콜 99.6% 감소**
- **프레임률 37 FPS → 92 FPS**

우당탕 방범대 | 핵심 구현 ④ VAT + GPU Instancing

Profiler와 Frame Debugger로 확인한 효과

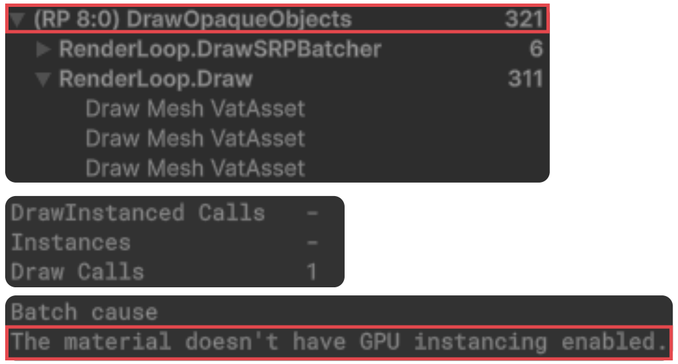
Left · Animator　Right · VAT　　Profile Analyzer · 마커별 300프레임 중앙값 (ms)

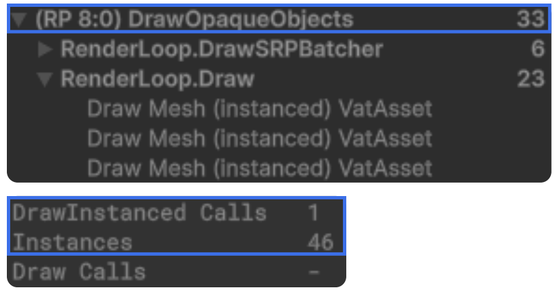
Marker Name	Left Medi	<	>	Right Med
PlayerLoop	26.37			11.03
Animators.Update	9.70			0.14
Director.ProcessFrame	9.68			0.14
WaitForJobGroupID	9.48			0.64
PreLateUpdate.DirectorUpdateAnimationEnd	6.93			0.09
Animators.WriteJob	5.38			0.01
PostLateUpdate.UpdateAllSkinnedMeshes	1.39			0.07
MeshSkinning.Update	1.36			0.07

VAT - Animator 갱신 9.70 → 0.14 ms

- Development Build · 1,000마리 · 렌더 스케일 0.3
- PlayerLoop(출력 대기 포함) 26.37 → **11.03 ms**
- 스키닝(MeshSkinning) 1.36 → **0.07 ms**

Instancing OFF　　Instancing ON (현재)　　Frame Debugger





GPU Instancing - 그리기 이벤트 321 → 33개

- 에디터 플레이 모드 · 몬스터 7종 300마리
- 몬스터는 종류별로 묶여 298번 → **10번**
- 드로우콜 1번에 같은 종류 **46마리** (Instances 46)